

Project Report – Database Systems

TourismSG

Team number: <59>

Student names: <Ryan Ang Rui Heng, Daryl Wong Le Jie, Kinston Lim

Kun Wei, Chua Jia Jun Jeston, Teng Xin Yi, Lim Jie Yi Valerie>

<29/11/2025 and 2359>

Submitted as part of the requirement for INF2003 Database Systems

1. Introduction

The rapid growth of tourism platforms has created a strong need for systems capable of handling high-volume structured information like hotels and attractions while simultaneously supporting flexible, real-time user generated content such as review and multimedia. Traditional relational databases excel in enforcing consistency and integrity, but struggle with schema flexibility. Conversely, NoSQL systems handle varying data formats efficiently but do not guarantee strong relational consistency.

To deepen our understanding of relational and non-relational database systems, the team designed and developed TourismSG, a full-stack tourism web application that demonstrates real-world database architecture, connectivity, and performance across two different DBMS paradigms. The project explores how SQL and NoSQL databases can coexist within a single application to deliver fast, reliable, and scalable data services.

TourismSG allows users to browse attractions, hotels, concerts, and food places, view detailed information, read reviews, and make bookings. The system also supports user authentication, protected dashboard features, and real-time review updates.

The project intentionally separates data responsibilities based on structure, consistency, and usage patterns:

- MySQL (Relational Database)

Used for highly structured, transactional, and consistency-critical data such as:

- Users, with hashed passwords
- Bookings for hotels and concerts
- Attraction, hotel, concert, food listings
- Admin and role-based access permissions

MySQL's strict schema design allowed us to practice database entity relationship modelling, normalization to 3NF, integrity constraints, foreign keys and optimized indexing

- Firebase Firestore (NoSQL Database)

Used for semi structured and real time data:

- User reviews
- Usage logs
- Image metadata or multimedia
- Future extensions such as notifications

To support this hybrid architecture, we implemented a modern full-stack environment using React(Vite) for the frontend and Node.js + Express for the backend. RESTful APIs serve as the communication bridge between the interface, the MySQL relational engine, and the Firebase NoSQL datastore. This architecture also allowed us to analyze challenges such as data synchronization between SQL and NoSQL.

We also worked with real datasets from Data.gov.sg, Kaggle, and tourism-related APIs. These

required significant cleaning to fix issues like inconsistent encoding, missing fields, schema mismatches, and broken image URLs.

2. System architecture Design and requirements

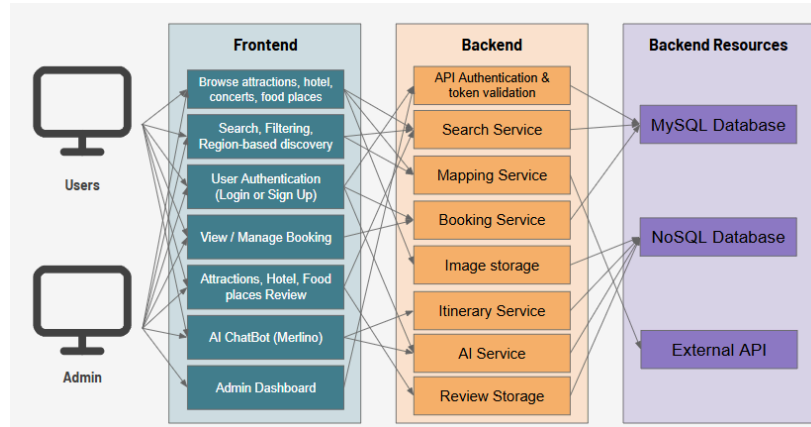


Figure 1: System architecture

Frontend:

- Browse attractions, hotels, concerts, food places
- Search, filtering, region based discovery
- User authentication, Login or Sign Up
- View and manage bookings
- Leave reviews, real time NoSQL updates
- AI Chatbot Interaction, Merlino and Itineraries
- Unified item detail pages with SQL + NoSQL data combined

Backend:

- SQL CRUD for hotels, attractions, bookings, user management, foodplaces
- NoSQL CRUD for reviews, AI Chat, Itineraries, logs and images
- API authentication and token validation
- Complex query support for indexed search, aggregates and joins
- Data merging across databases
- Error handling and validation

Database Layer:

- MySQL's strict schema, foreign keys and constraints
- Firestore's flexible document storage for reviews and logs

3. Relational Database Implementation (MySQL)

3.1 ER Diagram and Schema

The relational schema was normalised to Third Normal Form (3NF). This can be seen as all non key attributes depend solely on the primary key of their own table. The choice of using 3NF was to ensure logical consistency as well as eliminate redundancies across the database.

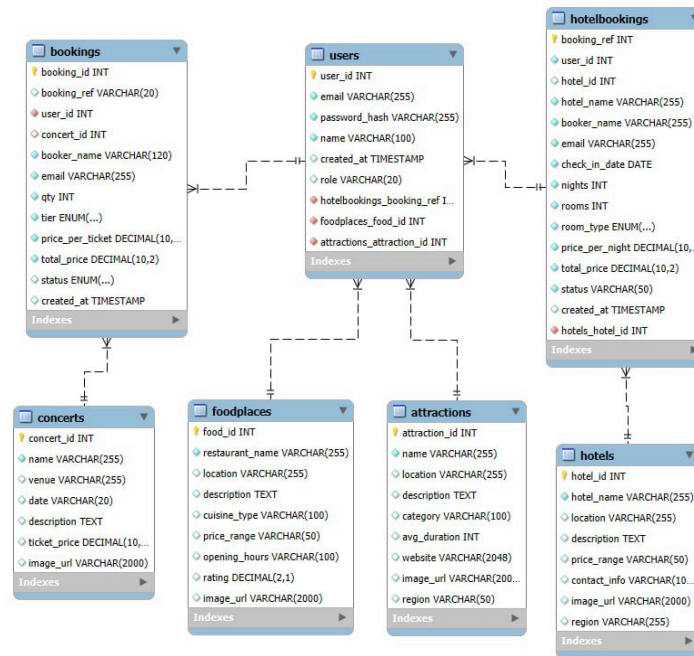


Figure 2: ER diagram

Table	Purpose	Reason for Separation
User	Manages authentication and user identity	Avoids duplication of user details across bookings; supports role-based access
Hotels	Stores hotel descriptions, regions, price ranges	Hotels require hotel-specific attributes; merging into attractions causes attribute sparsity
Attractions	Provides structured attraction information	Keeps datasets clean since attractions differ significantly from hotels/concerts
Concerts	Stores event schedules, venues, ticket prices	Events have time-sensitive attributes unsuitable for other listings
Foodplaces	Restaurant/cuisine information	Unique prices/cuisines justify separation
Bookings	Concert ticket reservations	Contains booking metadata linked with concerts and users
HotelBookings	Hotel stay reservations	Requires nights, room type, and room count fields unique from concert bookings

3.2 Backend Architecture and Relational Database Implementation (MySQL)

The backend is built using Node.js with Express and exposes a set of RESTful endpoints that mediate all access to the MySQL relational database. The React frontend never talks directly to MySQL; instead, it sends HTTP requests to the backend, which performs validation, executes SQL queries using prepared statements, and returns JSON responses. This separation improves security, maintainability, and testability of the system.

3.2.1 API Endpoints (SQL Backend)

Key endpoints include (non-exhaustive):

- GET /api/attractions – returns a list of attractions from the attractions table.

- GET /api/hotels – returns all hotels from the hotels table.
- GET /api/concerts – returns all concerts from the concerts table.
- GET /api/foodplaces – returns all food places from the foodplaces table.
- GET /api/:category/:name – returns detailed information for a single place (attraction, hotel, concert, or food place), used by DetailPage.jsx to render the detailed view.
- POST /api/bookings – creates a concert booking record.
- POST /api/hotelbookings – creates a hotel booking record.

3.2.2 Table Design and Relationships

The core relational schema is normalised to Third Normal Form (3NF) and includes the following key tables:

Table	Key Fields	Constraints	Justification
users	user_id (PK, AUTO_INCREMENT), name, email, password_hash, role	UNIQUE(email), NOT NULL	Prevent duplicate accounts; authentication
hotels	hotel_id (PK), hotel_name, region, location, price_range	NOT NULL indexed (region)	Faster regional filtering
concerts	concert_id (PK), name, venue, location, ticket_price, image_url, date	NOT NULL(name, venue, date), ticket_price >= 0, Composite indexing on (venue, date) improves upcoming event sorting	Stored separately because they require unique fields that would create many NULL values
attractions, foodplaces	*_id (PK), attributes	Optional fields allowed	Flexible content population
bookings (for concert)	booking_id (PK), user_id (FK), hotel_idconcert_id	FK with CASCADE rules	Ensure booking integrity
hotelbookings	booking_ref (PK), user_id (FK), hotel_id (FK), hotel_name, booker_name, email, check_in_date, nights, rooms, room_type, price_per_night, total_price, status, created_at	FK(user_id) ON DELETE CASCADE; FK(hotel_id) ON DELETE RESTRICT; required fields NOT NULL; status limited to valid booking states	Maintain referential integrity; prevent invalid bookings; retain price + hotel snapshot for historical and audit accuracy

3.2.3 CRUD Operations (Relational)

The backend exposes full CRUD capabilities:

- **Create** – e.g., concert bookings from DetailPage.jsx:
 1. `const r = await fetch(`${API}/api/bookings`, {`
 2. `method: "POST",`
 3. `headers: { "Content-Type": "application/json" },`
 4. `body: JSON.stringify({`
 5. `user_id: user.user_id,`
 6. `booker_name: user.name,`
 7. `email: user.email,`

```

8.   concert_id: data.concert_id,
9.   qty,
10.  tier,
11.  price_per_ticket: pricePer
12. })
13. });

```

- **Load (lists)** – e.g. load foodplaces and normalise data

```

1. useEffect(() => {
2.   const load = async () => {
3.     try {
4.       const res = await fetch(`${API}/api/foodplaces`);
5.       const raw = await res.json();
6.       const rows = Array.isArray(raw?.data) ? raw.data : raw;
7.
8.       // Normalize data: match SQL column names to frontend names
9.       const normalized = rows.map((f) => ({
10.        ...f,
11.        cuisine: f.cuisine || f.cuisine_type || "",
12.        price_range: f.price_range || f.price || "",
13.      }));
14.
15.      setFoodPlaces(normalized);
16.    } catch (err) {
17.      console.error("Error loading food places:", err);
18.    } finally {
19.      setLoading(false);
20.    }
21.  };
22.
23.  load();
24. }, [API]);

```

- **Read (details)** – We have a details page that uses a universal fetch function to load in data for any category

```

1. useEffect(() => {
2.   (async () => {
3.     try{
4.       const r = await fetch(`${API}/api/${category}/${encodeURIComponent(name)}`);
5.       if(!r.ok) throw new Error("Failed to fetch details");
6.       const json = await r.json();
7.       const unified = {
8.        ...json,
9.        title: json?.name || json?.hotel_name || json?.restaurant_name || name,
10.        location: cleanText(json?.location || json?.venue || ""),
11.        region: cleanText(json?.region || ""),
12.        description: cleanText(json?.description || json?.long_desc || ""),
13.        website: json?.website ? cleanText(json.website) : null,
14.        image_url: json?.image_url || json?.image || null,
15.      };
16.      setData(unified);
17.    } catch(e){ console.error(e); }
18.    finally { setLoading(false); }
19.  })();
20. }, [API, category, name]);

```

- **Update / Delete** – implemented in admin-only endpoints (not shown in the React files) to allow editing and removal of listings. These use authenticated routes and prepared statements.

3.2.4 Advanced SQL Features

- Prepared Statements for all SQL operations to prevent SQL injection.


```
1. SELECT * FROM hotels WHERE region = ? AND hotel_name LIKE CONCAT('%', ?, '%');
```
- JOIN Queries to retrieve enriched booking information (user + hotel/concert) in a single query, used for dashboards

1. SELECT b.booking_id, u.name AS user_name, c.name AS concert_name, b.qty, b.total_price
 2. FROM bookings b
 3. JOIN users u ON b.user_id = u.user_id
 4. JOIN concerts c ON b.concert_id = c.concert_id
 5. WHERE u.user_id = ?;
- Indexes to improve performance on frequent search/filter fields
 1. CREATE INDEX idx_hotels_region_name ON hotels (region, hotel_name);
 2. CREATE INDEX idx_attractions_region_category ON attractions (region, category);
 - Constraints
 1. UNIQUE(email) on users to prevent duplicate accounts
 2. NOT NULL on critical fields (password_hash, user_id, foreign keys)
 3. CHECK(total_price >= 0) on booking tables (if supported by the engine)
 - Normalization to 3NF ensures that all non-key attributes depend on the primary key and not on each other. Hotels, concerts, attractions, and food places each have their own dedicated tables; bookings are stored separately and only reference those tables via foreign keys. This prevents duplication of place details inside booking records.

4. Non-Relational Database Implementation

4.1 NoSQL collection and Design

Our NoSQL (Firestore) implementation operates independently from the SQL (MySQL) implementation. The separation between the 2 databases gives us more flexibility and real-time responsiveness. By keeping them separated, we can design the NoSQL database to be more user-generated content rather than forcing it to follow strict relational schema like MySQL. This allows features such as review, image and AI chat log to evolve freely without the need to modify existing tables or risk creating schema conflicts.

Firestore's document-based models are able to perform hierarchical data, fast writes and live updates, and relations databases have hard time to achieve them efficiently. To take full advantage of this, our NoSQL is designed based on how the application interacts with the data, making it fully independent from the SQL schema.

Although there are tools and systems that allow us to convert relational schemas into NoSQL structures, they are not required for our system as our databases are intentionally designed to handle different workloads. While the SQL maintains structure, transactional records, NoSQL handles the dynamic, user driven content. This setup will ensure our system remains scalable, maintainable, and adaptable to future features added in.

4.2 Collection and Document Structure

- reviews
Stores user reviews for any place
 1. {
 2. "placeName": "Marina Bay Hotel",
 3. "userId": "123",
 4. "userName": "Alice",
 5. "rating": 5,
 6. "comment": "Great view and service!",
 7. "timestamp": 1735849266
 8. }
- logs
Application event logs
 1. {
 2. "userId": "123",
 3. "action": "create_booking",
 4. "context": "concert",
 5. "timestamp": 1735849300

- 6. }
- **images**
Metadata for images associated with places
 1. {
 2. "itemId": "hotel_45",
 3. "url": "https://...",
 4. "altText": "Night view of the hotel",
 5. "uploadedAt": 1735849350
 6. }

This design is independent of the SQL schema; Firestore does not mirror the relational tables directly. Instead, each collection is designed from scratch based on usage patterns, such as frequent small writes and unstructured text fields.

4.3 NoSQL CRUD via ReviewBox, Chatbot and itinerary Generator

The DetailPage.jsx component renders a ReviewBox Child, passing placeName so that Firestore can load and store reviews associated with that place

```
1. <ReviewBox placeName={data.title} />
```

Inside ReviewBox, Firestore is used to:

- **Create** reviews with addDoc(collection(db, "reviews"), {...})
- **Read** reviews with a query filtered by placeName
- Optionally **update/delete** reviews for editing features

The AI Chatbot (Merlino) also uses Firestore to maintain user-specific chat history inside the chatlogs collection.

Firestore is used to:

- Create chat session with addDoc(collection(db, "chatlogs"), {...}) (JSON Document)
- Read chat logs in real-time using query and snapshot
- Update chat messages with updateDoc(doc(db, 'chatlogs', chatId), {...})
- Delete chat history with deleteDoc(doc(db, "chatlogs", chatID))

The Itinerary system manages user-created trips as well as the AI Generated created trip using the Firestore itineraries collection.

Firestore is used to:

- Create itineraries through the form or AI generator with addDoc(collection(db, "itineraries"), {...})
- Read existing itineraries with getDocs(query(collection(db, "itineraries"))
- Update itinerary days/activities, including editing times or adding items
- Delete itineraries from the planner

This satisfies the basic NoSQL CRUD requirements.

4.4 Advanced NoSQL Features

- **Automatic Indexing and Composite Indexes**
Firestore automatically indexes documents by ID and can be configured with composite indexes on fields such as (placeName, timestamp) to support queries like "all reviews for this place ordered by newest first".

- **Real-time Capability (Extensible)**

While the current implementation typically issues one-off fetches, the system can be extended to use onSnapshot listeners, enabling reviews to update live without page refreshes. Firestore's real-time infrastructure and offline cache are already supported by the chosen client SDK and architecture.

- **Loose Coupling With SQL**

The backend and frontend treat Firestore as an independent data source. There is no requirement that each SQL row has a corresponding Firestore document. This loose coupling is an advanced architectural technique that maintains flexibility while preserving transactional ACID logic within SQL

4.5 Integration of SQL and NoSQL in the Backend

A key design goal was to merge SQL item data and NoSQL review data into a single, coherent user experience.

1. The backend (or frontend, via separate calls) fetches place details from MySQL
 1. `const r = await fetch(`${API}/api/${category}/${encodeURIComponent(name)}`);`
 2. `const json = await r.json();`
2. The DetailPage constructs a unified object for display, normalising fields like title, location, region, description and website.
3. ReviewBox queries Firestore for reviews with the same placeName and renders them under the SQL-based details section, giving the user a seamless view of both structured (SQL) and unstructured (NoSQL) data on a single page.

This architecture demonstrates the core learning outcome of the module: designing and implementing an application that uses both a relational database and a non-relational database, and integrating them in a meaningful way.

5. Discussions

The system was built with the inherent goal of supporting the demands of a modern day tourism platform while balancing SQL's strict consistency with NoSQL's flexibility. During development, the architecture was evaluated on how well it performed in terms of scalability, reliability, security, and overall behaviour under different workloads. The hybrid between SQL and NoSQL setup was proven useful, but also showed limitations on areas where the system can be improved.

5.1 Scalability

The system is structured to scale horizontally without rewriting core logic. Several components support growth, although further optimisation may still be needed. These are the design decisions that contribute directly to scalability:

- **Stateless React SPA:** Keeps the client completely stateless, allowing multiple front-end instances to run without conflict.
- **REST API:** Enables backend duplication across multiple servers, allowing routing and load balancing without modifying client logic.
- **Firebase Firestore:** Automatically handles scaling for high read/write volumes.

- **Indexed MySQL fields:** (e.g., region, category, venue-date) improves lookup times significantly.

This architecture is able to serve a moderate number of concurrent users, especially for read heavy workloads. Write heavy SQL operations may eventually require replication or caching but the base design is robust and scalable enough to support the projected system growth without major architectural changes.

5.2 Reliability

MySQL provides full ACID properties, ensuring that booking transactions are atomic and consistent. This prevents partial or corrupted records and maintains strong reliability for structured data.

- **Try/catch wrappers:** Handle failed requests gracefully instead of crashing the UI.
- **SQL integrity constraints:** Prevent invalid or incomplete booking records. (NOT NULL, UNIQUE)
- **Firestore's internal replication:** Ensures user content remains accessible and synchronised even with network issues.

Even with inconsistent public datasets, the application maintains a stable behaviour and avoids any major data corruption or UI failures, ensuring a reliable experience for end users.

5.3 Security Features

Security was implemented across multiple layers, preventing common vulnerabilities and ensuring that restricted actions cannot be performed by unauthorised users.

- Password hashing (bcrypt) ensures credentials are not stored in plain text.
- AuthContext enforces identity state on the frontend.
- Protected routes restrict access for non-authenticated and non-admin users.
- Prepared SQL statements guard against injection.
- Firebase security rules ensure only authenticated users can write reviews or logs.

Together, these security features provide a layered defence against common web vulnerabilities such as credential theft and unauthorised data manipulation while ensuring that user data and transactional actions remain protected.

5.4 Performance Analysis

Performance testing across SQL and NoSQL operations (Figure 3) has shown that the hybrid model offers balanced speed for both structured reads as well as dynamic user generated content.

SQL Performance (MySQL):

- **Read operations:** ~25-60ms per query (indexed)
- **Write operations:** ~30-50ms for bookings
- **JOIN operations:** slightly higher latency but still consistently stable

Firestore Performance (NoSQL):

- **Writes:** ~8-15ms
- **Reads (cached):** ~5-10ms
- **Real time:** Instant update propagation for reviews

These results demonstrate that SQL is able to handle structured and JOIN heavy queries effectively while Firestore performs better in terms of low latency writes and dynamic content. Thus, this hybrid combination optimises performance across different workloads.

5.5 Limitations

Because Firestore and MySQL do not share ACID transactions across systems, true cross-database atomic commits are not possible. This may lead to minor inconsistencies in scenarios where SQL and Firestore updates must happen together.

- **No cross database transactions:** SQL and Firestore cannot share atomic commitment, which may result in inconsistencies.
- **Data quality issues:** Public datasets contain missing values and inconsistent encodings that require manual cleaning, especially for images.
- **No dedicated caching layer:** Heavier SQL reads might benefit more from Redis caching
- **Limited admin tools:** Admin CRUD endpoints exist, but the frontend for managing them is still little.

5.6 Future Considerations

To improve scalability, maintainability, and user experience, several enhancements can be explored and implemented:

- **Implement microservers:** To separate bookings, listings, reviews, and authentication modules.
- **Add Redis caching:** For frequently accessed SQL queries
- **Add Cloud Functions:** For event-driven notifications (e.g. booking confirmations)
- **Expand NoSQL usage:** To store chat logs and booking activity analytics
- **Improve admin dashboard:** Include data analytics (e.g. firestore activity tracking)

6. Reflection and Takeaways

This project significantly strengthened our understanding of database system design, hybrid SQL/NoSQL modelling, and full-stack development. Working with two different database paradigms forced us to consider trade-offs between consistency, flexibility, and maintainability. Through the implementation of MySQL, we gained hands-on experience with relational schema design, normalisation to 3NF, indexing, and ER modelling. At the same time, using firestore allowed us to explore document based modeling, schema-less design and real time data behaviour which SQL systems cannot provide efficiently. Integrating these two databases into one unified detail page required careful API design, data cleaning and frontend normalisation which helped improve our understanding of multi database coordination.

Collaboration within the team has also strengthened our version control discipline, debugging techniques, and the ability to translate different theoretical principles into a singular working system. This project has helped highlight to us the strengths and limitations of each database model and gave us valuable insights which we can apply for future modules and projects.

Reference

The references are a list of any sources you have used in your report. Your report should use the standard referencing style following an established referencing format (e.g., IEEE, Harvard, APA, etc.).

Appendix I

Download link of the source code: <https://github.com/DarylW3103/TourismSG>

to start the AI Chatbot(AI_service) please do `py train_intent.py` first then `py ai_service.py`
 to start the backend(Toursim_backend), do `npm install` first, follow by `node server.js` (please also edit the [db.js](#) to your own password, as well as edit the port number to your own db port number e.g 3006)
 to start the frontend(Tourismboored), do `npm install` first, follow by `npm start`

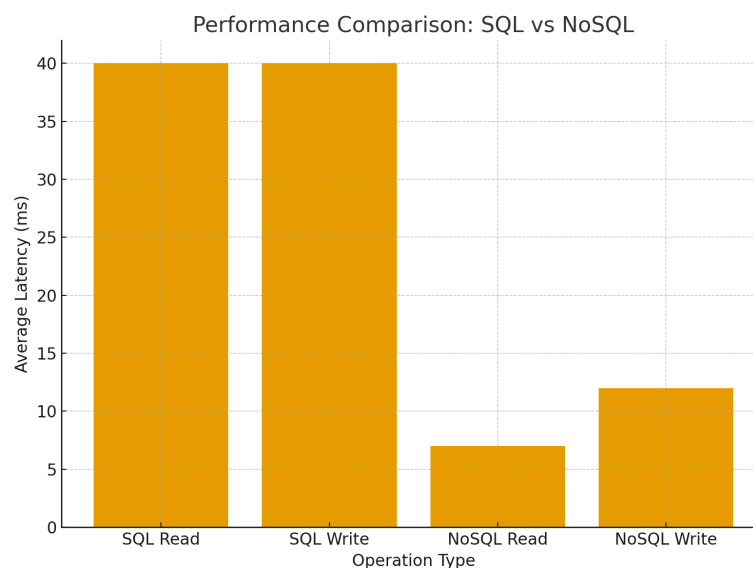


Figure 3: Performance Comparison

END OF REPORT